

Annexe 4 : Rédaction de la documentation technique et déploiement

1. Contexte technique et documentation	2
2. Architecture de la documentation	2
A. Automatisation du lancement	2
B. Indication des prérequis	2
C. Sécurité et configuration	2
D. Gestion des erreurs de saisie	3
E. Structure et arborescence	3
3. Preuves matérielles	3
4. Bilan	6

1. Contexte technique et documentation

Après avoir terminé le développement des outils d'extraction (**RET** et **ECM PDF Getter**), il fallait préparer le déploiement. Tout logiciel professionnel a besoin d'une notice claire pour expliquer son fonctionnement, et c'est une norme de livraison indispensable. De plus, des actions spécifiques sont requises (configuration manuelle des identifiants par exemple), j'ai donc rédigé un tutoriel d'utilisation (une par application) afin d'expliquer la marche à suivre étape par étape et de garantir la bonne exécution du script.

2. Architecture de la documentation

A. Automatisation du lancement

Pour éviter de faire utiliser des commandes complexes dans la console, j'ai créé un fichier **start.bat**. L'utilisateur aura juste à cliquer dessus. Le script télécharge et installe les **bibliothèques requises** automatiquement, puis il lance automatiquement le fichier **python** principal.

B. Indication des prérequis

Le programme nécessite malgré tout des prérequis qui devront être appliqués par l'utilisateur, comme le système d'exploitation et le langage de programmation (**python**) pour les 2 programmes (**Documentum** et **Adobe** pour le **ECM PDF Getter**) ils sont indiqués au début du **README.txt**

C. Sécurité et configuration

Le fichier de documentation **README.txt** explique comment remplir le fichier **config.json**. On demande à l'utilisateur de taper les identifiants directement dans le fichier (pour des raisons de sécurité). De cette façon, les identifiants ne sont jamais écrits en dur dans le code source ou dans la console.

D. Gestion des erreurs de saisie

Il est indiqué qu'il faut potentiellement enlever les guillemets des chemins, Ils peuvent apparaître lorsqu'on glisse un chemin dans la console. **Windows** peut ajouter des guillemets lorsqu'un nom de fichier ou dossier contient des espaces.

Il est aussi indiqué que l'utilisateur peut effectuer un arrêt d'urgence avec une combinaison de touches (<Ctrl>+<Shift>+<Q>). J'ai ajouté cette méthode de sécurité pour pouvoir forcer l'arrêt du programme (spécifiquement pour le **ECM PDF Getter** étant donné qu'il peut bloquer la souris).

E. Structure et arborescence

La structure du projet est indiquée, elle est très importante pour montrer quels sont les fichiers et dossiers nécessaires pour la bonne exécution du programme

3. Preuves matérielles

Description et prérequis (lien masqué pour des raisons de confidentialité) :

```
## Description
Ce programme permet de récupérer des PDF via des liens objet ECM pour ensuite les stocker dans un répertoire défini par l'utilisateur.
Le programme scanne le(s) fichier(s) Excel (.xlsx ou .xslm) pour extraire les liens cibles.
Exemple : https://[REDACTED]/<ID_DE_L'OBJET>/[REDACTED]
Le but du programme est de les télécharger à la chaîne et de les ranger.

## Prérequis
- Python
- Système d'exploitation : Windows 11.
- Environnement : Accès fonctionnel à Documentum.
- Adobe Acrobat
```

Installation (après le lien, il est écrit "-r bibliothèque_requis.txt") :

```
## Installation
Cliquez sur start.bat.
Le programme va automatiquement :
- Installer les bibliothèques requises (via "pip install --index-url https://[REDACTED]")
- Lancer l'exécution du script principal ("RET_PDF_Getter.py").
```

Figure 1 : Capture de la section "Description", "Prérequis" et "Installation".

Consignes mentionnant la suppression des guillemets (si présent) :

```
Il vous demandera de :
- Configurer le chemin du dossier où seront stockés les dossiers suivants :
  - "RET_PDF"
  - "old_RET_PDF"
  - "RET_Logs"

Le programme attendra un chemin valide qui pointe vers un dossier.
Vous pouvez glisser le dossier directement dans la console pour qu'il imprime son chemin.
(Pensez à enlever les guillemets s'ils sont présents).

Si le chemin est valide mais qu'il n'existe pas, le programme vous demandera si vous souhaitez créer un dossier à l'emplacement.
Si vous êtes en train de reconfigurer, vous pouvez appuyer sur "Entrée" (à vide) pour garder la config actuelle.

- Configurer le(s) chemin(s) des fichier(s) Excels (.xlsx ou .xlsm) où seront stockés les numéros des objets.

Le programme attendra un chemin valide qui pointe vers un fichier Excel (.xlsx ou .xlsm).
Vous pouvez glisser le fichier directement dans la console pour qu'il imprime son chemin.
(Pensez à enlever les guillemets s'ils sont présents).

Vous pouvez :
- Entrez un nouveau chemin pour ajouter un fichier à la liste.
- Tapez "/RESET" pour réinitialiser la liste actuelle.
- Appuyez sur "Entrée" (à vide) pour confirmer (Vous devez avoir au moins 1 chemin enregistré).
```

L'arrêt d'urgence est indiqué dès le début des consignes d'utilisations :

```
## Utilisation

Vous pouvez forcer l'arrêt du programme en appuyant simultanément sur <Ctrl>+<Shift>+<Q>.

La première fois :
  Le programme va automatiquement créer un fichier config.json à la racine du programme.
  Il vous demandera d'inscrire un NNI et le mot de passe SESAME directement dans le fichier config.json.
  Le programme s'arrêtera ici.
  Vous pouvez le redémarrer après avoir renseigné les identifiants.
```

Figure 2 : Capture de la consigne sur les guillemets et l'arrêt d'urgence.

Listes des technologies utilisées (contient aussi les bibliothèques) :

```
## Technologies utilisées
- Langage : Python
- Automatisation Web & Requêtes : Selenium, Documentum, requests, urllib3
- Navigateur embarqué : Google Chrome (for Testing)
- Manipulation de données : openpyxl (Excel), JSON (Configuration)
- Interactions utilisateur : pynput (arrêt d'urgence clavier), inputtimeout (gestion des délais console)
```

Figure 3 : Capture de la liste des technologies utilisées à la fin du document.

Structure du projet:

```
## Structure du projet

|- chrome_env
|---|- chromedriver-win64
|    |- chrome-win64
|    |- extension
|
|- src
|---|- ConfigManager.py
|    |- ConfigQuestions.py
|
|- config.json (le programme le créera de lui-même)
|
|- RET_PDF_Getter.py
|
|- bibliothèque_requis.txt
|
|- start.bat
|
|- README.txt
```

Figure 4 : Capture de l'arborescence et de la structure du projet, indiqué à la fin du document.

4. Bilan

L'objectif pratique est atteint et l'utilisateur est complètement autonome du début à la fin. Le document décrit toutes les actions du programme, comme la suppression des vieux fichiers et le déplacement dans le dossier de sauvegarde. Comme ça, la personne sait exactement ce que le script fait sur sa machine et ne panique pas pendant l'exécution. J'ai listé les composants (comme **Selenium** ou **Google Chrome for Testing**) dans le fichier pour qu'il sache ce qui est téléchargé et utilisé sur son poste de travail.